**ЛИДЕРЫ ЦИФРОВОЙ ТРАНСФОРМАЦИИ · 2026**

# ФСП Таланты

Цифровая платформа подбора IT-специалистов с обратной механикой найма и верифицированным профилем достижений участника Федерации спортивного программирования

**Сопроводительная документация к решению** (ТЗ, раздел 3.4)

Команда: **KARMA TEAM**

Версия API: **1.4.0** · миграция БД **0006** · ветка **main**, коммит **c19f7d3**

Репозиторий: **github.com/NedYouNet/LLH_BY_KARMA**

Развёрнутый стенд: **https://lct.demo.bloomee.ru** · демо-почта: **https://mail.lct.demo.bloomee.ru**

Дата: 11 октября 2026

# Содержание


1 О решении и карта соответствия ТЗ 3

2 Функциональная архитектура 5

3 Компонентная архитектура 7

4 Механика тестирования и обоснование устойчивости 10

5 Механика подбора и логика категоризации 14

6 Процедура валидации и её результаты 16

7 Схема интеграции с реестром ФСП и ФСП ID 19

8 Описание API 21

9 Библиотеки и компоненты с версиями 28

10 Сборка, развёртывание и локальный запуск 30

11 Безопасность и 152-ФЗ 33

12 Использование ML и LLM 35

13 Дополнительный функционал: отклики, задания, ATS, защита от фиктивных вакансий, мобильная версия 36

14 Ограничения и планы развития 38

П Приложения: демо-данные, переменные окружения 39

# 1 О решении и карта соответствия ТЗ

**«ФСП Таланты»** — веб-платформа подбора IT-специалистов с обратной механикой: работодатель сам находит кандидата в уже категоризированной базе и направляет ему адресное приглашение с вилкой зарплаты. Категория кандидата «направление · грейд» присваивается не по резюме, а по результату обязательного пути: анкета → самостоятельный выбор грейда → практический тест из трёх задач на Python. Достижения ФСП подтягиваются в профиль через вход по ФСП ID и поднимают кандидата в выдаче внутри категории. Контакты кандидата закрыты до тех пор, пока он не примет приглашение или не откликнется сам. В интерфейсе продукт называется «Карьерная платформа ФСП», API — «FSP Talent API».

**20**
категорий: 5 направлений × 4 грейда

**84**
метода REST API, OpenAPI 3.1

**182**
автотеста: SQLite и PostgreSQL, все зелёные

**0.365**
P@10 подбора против 0.185 у поиска по ключевым словам

## Соответствие обязательному перечню документации (ТЗ, п. 3.4)

| Требование ТЗ | Где в документе |
|---|---|
| Функциональная и компонентная архитектура решения | Разделы 2 и 3 |
| Механика тестирования с обоснованием устойчивости к распространению заданий | Раздел 4 (п. 4.7 — обоснование, п. 4.8 — честные ограничения) |
| Механика подбора и логика категоризации | Раздел 5 |
| Собственная процедура валидации и её результаты | Раздел 6 |
| Схема предполагаемой интеграции с реестром ФСП | Раздел 7 |
| Описание API | Раздел 8 (полная таблица 84 методов) |
| Библиотеки и компоненты с указанием версий | Раздел 9 |
| Пошаговая инструкция по сборке, развёртыванию и запуску | Раздел 10 |

## Соответствие функциональным требованиям ТЗ (раздел 2)

| Требование | Как реализовано |
|---|---|
| Обязательный путь при регистрации: опрос → выбор грейда → тест | Анкета ( `POST /api/candidate/survey` ) записывает только заявленные направление и грейд; категорию присваивает только завершённый тест (раздел 4) |
| Обратная механика: работодатель сам выходит на кандидата | Подборка по категориям с обоснованием, адресное приглашение с обязательной вилкой ЗП, статусы *отправлено → просмотрено → принято/отклонено/ отозвано* |
| Контакты скрыты до принятия приглашения или отклика | Правило `contact_basis` на сервере; журнал доступа; отзыв доступа кандидатом (раздел 11) |

| Требование | Как реализовано |
|---|---|
| Зарплата обязательна в вакансии и в приглашении, подписаны gross/net | Проверка в схемах API и ограничение `CHECK` в БД; поле `salary_type` («до вычета НДФЛ» / «на руки») |
| Корректная обработка профиля без истории ФСП | Кандидат без ФСП остаётся в выдаче; компонента ФСП равна 0, причина «Нет истории ФСП — оценка по тесту» |
| Грейд не по ФСП и не по вакансии (уточнение Q&A) | ФСП влияет только на ранжирование внутри категории; грейд меняется только по итогам теста |
| Неподтверждённый грейд показывать со статусом и опускать | Пометка «Грейд заявлен кандидатом», множитель 0.6 в рейтинге, фильтр «Только с подтверждённым грейдом» |
| Ограничение частоты смены грейда | Смена грейда — раз в 90 дней; пересдача того же уровня — раз в 24 часа; провал не понижает грейд |
| Стандартизированный PDF-профиль и загрузка PDF-резюме | `GET /api/candidate/profile/pdf`; разбор резюме GigaChat с резервным парсером (раздел 12) |
| Дополнительный функционал (критерий 6, 7 %) | Вакансии и отклики, короткие задания с оценкой, интеграция с ATS по вебхукам с подписью, концепция защиты от фиктивных вакансий, мобильная вёрстка (раздел 13) |

# 2 Функциональная архитектура

Платформа обслуживает две роли: **кандидат** и **работодатель**. Обе роли работают в одном веб-приложении; набор экранов и доступных методов API определяется ролью из JWT-токена. Ниже — сквозные пользовательские сценарии и функциональные модули.

## 2.1 Сквозной путь кандидата

**1. Регистрация**
email + пароль, согласие на обработку ПДн; подтверждение почты. Или вход через ФСП ID

→

**2. Профиль**
вручную или из PDF-резюме (GigaChat заполняет пустые поля)

→

**3. Анкета**
направление, опыт, стек, роль, формат; заявленный грейд

→

**4. Тест**
3 задачи на Python, 30 мин, до 10 проверок на задачу

→

**5. Категория**
«Бэкенд · Middle», история грейда, PDF-профиль

→

**6. Предложения**
приглашения с ЗП: принять / отклонить; вакансии и отклики; задания

## 2.2 Сквозной путь работодателя

**1. Компания**
описание, направление, контакт, ИНН (метка доверия)

→

**2. Потребность**
направление, грейд, стек, задачи команды

→

**3. Подборка**
категории, рейтинг с причинами и вкладом факторов, фильтры

→

**4. Приглашение**
текст, обязательная вилка ЗП, способ связи

→

**5. Статусы**
отправлено → просмотрено → принято / отклонено

→

**6. Контакт**
контакты открываются после принятия; событие в ATS

## 2.3 Функции кабинетов

| Вкладка кандидата | Что делает |
|---|---|
| Моя категория | текущая категория и балл, смена грейда (на какие уровни можно пройти тест и когда), история грейда и попыток |
| Профиль | анкетные данные, контакты, навыки, загрузка PDF-резюме |
| Анкета и тест | опрос и прохождение теста с редактором кода |
| Приглашения | входящие предложения с вилкой ЗП; принять, отклонить, закрыть контакты |

| Вкладка работодателя | Что делает |
|---|---|
| Компания | профиль компании, ИНН, способ связи |
| Потребности | сохранённые описания «кого ищем»; запуск подборки |
| Поиск кандидатов | вся база и избранное; фильтры; карточки с обоснованием; приглашение |
| Вакансии | публикация и редактирование, подборка под вакансию |

| Вкладка кандидата | Что делает |
|---|---|
| Вакансии | лента вакансий с фильтрами, отклик |
| Отклики | мои отклики и их статусы, отзыв отклика |
| Короткие задания | задания работодателей для моей категории |
| Согласия и ФСП | согласия по 152-ФЗ, приватность, привязка ФСП ID, журнал доступа, выгрузка, PDF, удаление аккаунта |

| Вкладка работодателя | Что делает |
|---|---|
| Отклики | входящие отклики: просмотрено / принять / отклонить |
| Короткие задания | создание заданий, просмотр и оценка решений 0..10 |
| Приглашения | отправленные приглашения и их статусы; контакты после принятия; отзыв |
| Интеграции | вебхук в ATS: адрес, секрет подписи, тестовое событие, журнал доставок |

## 2.4 Функциональные модули

| Модуль | Ответственность | Где в коде |
|---|---|---|
| Аутентификация | регистрация, подтверждение email, вход, ротация refresh-токенов, выход, блокировка перебора, вход через ФСП ID | `auth_service`, `fsp_id_client` |
| Профиль кандидата | профиль, анкета, согласия, приватность, разбор резюме, привязка ФСП | `candidate_service`, `ml_client` |
| Тестирование | генерация вариантов, проверка кода, подсчёт баллов, решение по грейду, кулдауны | `testing_service`, `testing_engine`, `code_runner` |
| Подбор | пул по категориям, фильтры, скоринг, объяснения, потребности, избранное | `matching_service`, `matching_engine` |
| Приглашения | машина состояний приглашения, контакты, отзыв доступа, событие в ATS | `invitation_service`, `presenters` |
| Вакансии и отклики | CRUD вакансий, лента, отклики и их статусы | `vacancy_service` |
| Короткие задания | задания работодателя, решения, оценки, влияние на активность | `short_task_service` |
| ФСП | реестр результатов (таблица + демо), балл ФСП, имитация ФСП ID | `fsp_service`, `fsp_data`, `fsp_id_mock` |
| Приватность | журнал доступа к контактам, выгрузка данных, удаление аккаунта | `privacy_service` |
| Интеграции | вебхуки ATS с подписью HMAC, тестовый приёмник, письма, PDF-профиль | `ats_service`, `email_service`, `pdf_service` |

# 3 Компонентная архитектура

Решение — монолит с явным разделением слоёв (разрешено Т3, п. 3.5), упакованный в четыре Docker-контейнера. ML-модули работают внутри процесса бэкенда; GigaChat вызывается только для разбора резюме и только при наличии ключа.

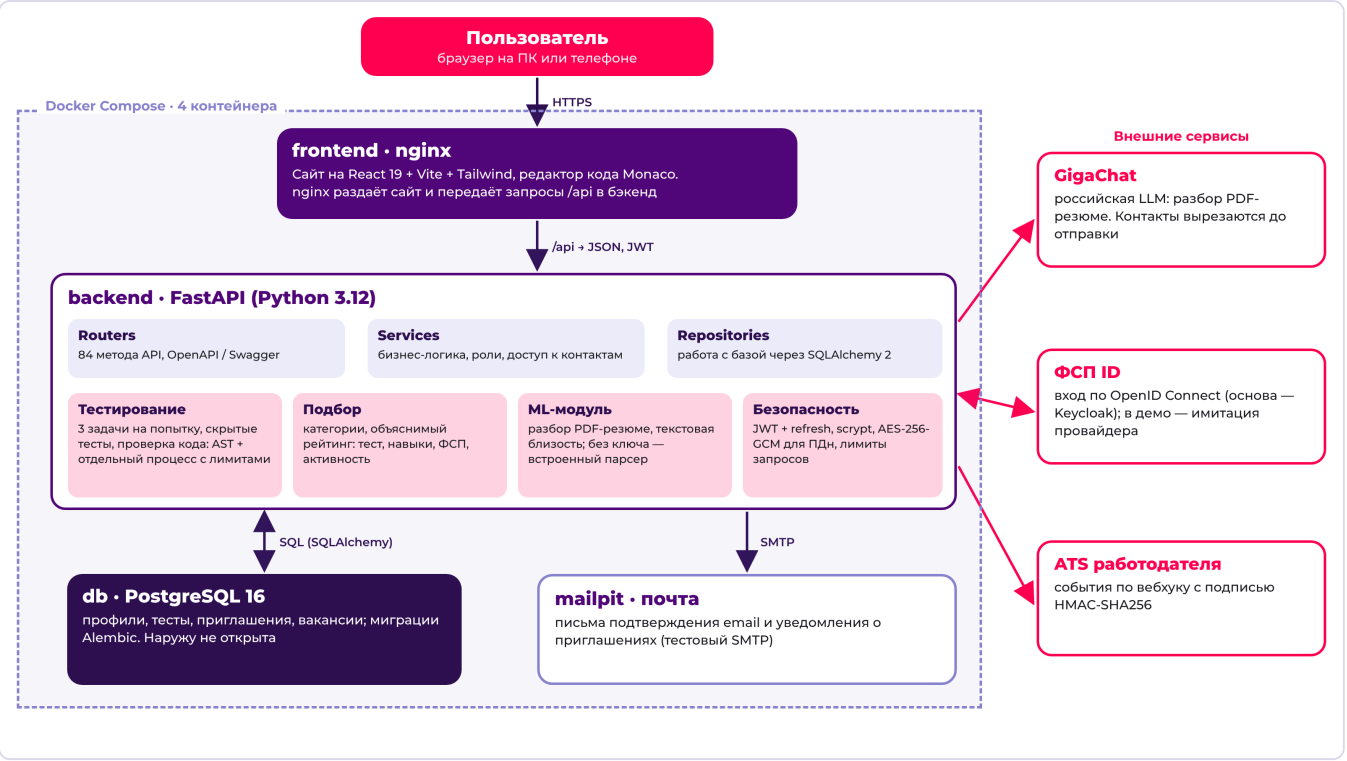
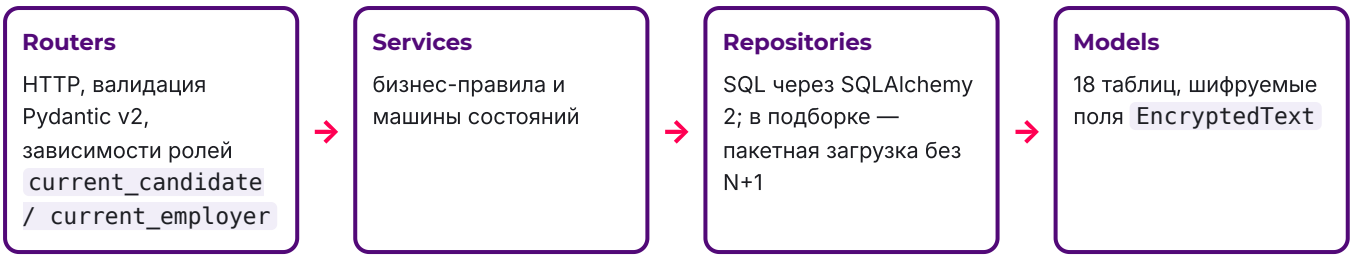



Рис. 1. Компонентная схема: контейнеры Docker Compose, слои бэкенда и внешние сервисы

## 3.1 Контейнеры

| Контейнер | Образ | Назначение | Порт |
|---|---|---|---|
| frontend | сборка `node:22-alpine` → `nginx:1.27-alpine` | раздаёт собранное React-приложение; проксирует `/api/` в backend; SPA-fallback; лимит загрузки 10 МБ | 127.0.0.1:5173 → 80 |
| backend | `python:3.12-slim`, пользователь `appuser` (не root) | FastAPI; при старте выполняет `alembic upgrade head`, затем uvicorn с `WEB_CONCURRENCY` процессами (по умолчанию 4) | 127.0.0.1:8000 |
| db | `postgres:16-alpine` | основное хранилище; healthcheck `pg_isready`; том `pgdata` | наружу не публикуется |
| mailpit | `axllent/mailpit` | тестовый SMTP и веб-интерфейс писем (подтверждение email, уведомления о приглашениях) | 127.0.0.1:8025 |

## 3.2 Слои бэкенда




Отдельно выделены **презентеры** (`presenters.py`) — превращают модели в ответы API с учётом прав доступа (раскрытие контактов, сокращение имени до «Имя Ф.»), и **движки без БД**: `matching_engine` (скоринг и объяснения), `testing_engine` (генерация и оценка теста), `code_runner` (проверка кода). Модели БД наружу не отдаются — контракт API задают схемы Pydantic.

## 3.3 Фронтенд

- React 19 + Vite 8 + Tailwind CSS 4, редактор кода Monaco (собирается локально, без CDN). Роутера нет: экран — состояние приложения; обрабатываются ссылки `?token=`, `/candidate/invitations`, `/auth/fsp-id`.
- Единый клиент API `api.js`: Bearer-токен в каждом запросе, обновление токена одним запросом даже при нескольких вкладках (Web Locks), единая модель ошибок `ApiError` с ошибками полей и подсказкой `Retry-After`.
- Светлая и тёмная тема, адаптивная вёрстка под телефоны (раздел 13.5), атрибуты доступности `aria-*`, видимый фокус.

## 3.4 Модель данных

18 таблиц предметной области, схема управляется миграциями Alembic `0001 → 0006`. Статусы хранятся строками. Удаление пользователя каскадно удаляет связанные данные; журнал безопасности `auth_events` сохраняется обезличенно (ссылка на пользователя обнуляется).

| Группа | Таблицы |
|---|---|
| Пользователи и безопасность | `users`, `refresh_tokens` (хранится только идентификатор токена), `auth_events` (журнал входов; email только как HMAC-отпечаток) |
| Кандидат | `candidate_profiles` (контакты и текст резюме зашифрованы), `grade_history`, `test_attempts` (задания с эталонами — только на сервере), `question_stats` |
| Работодатель | `employer_profiles` (контакты и секрет ATS зашифрованы), `employer_needs`, `shortlist_items`, `ats_deliveries`, `mock_ats_events` |
| Взаимодействие | `invitations` (CHECK на вилку ЗП, снимок обоснования подбора), `vacancies` (CHECK на вилку ЗП), `applications` (уникальная пара кандидат–вакансия), `contact_access_log` |
| Короткие задания | `short_tasks`, `short_task_submissions` (одно решение на задание от кандидата) |

## 3.5 Ключевой сценарий «приглашение → контакт» по компонентам

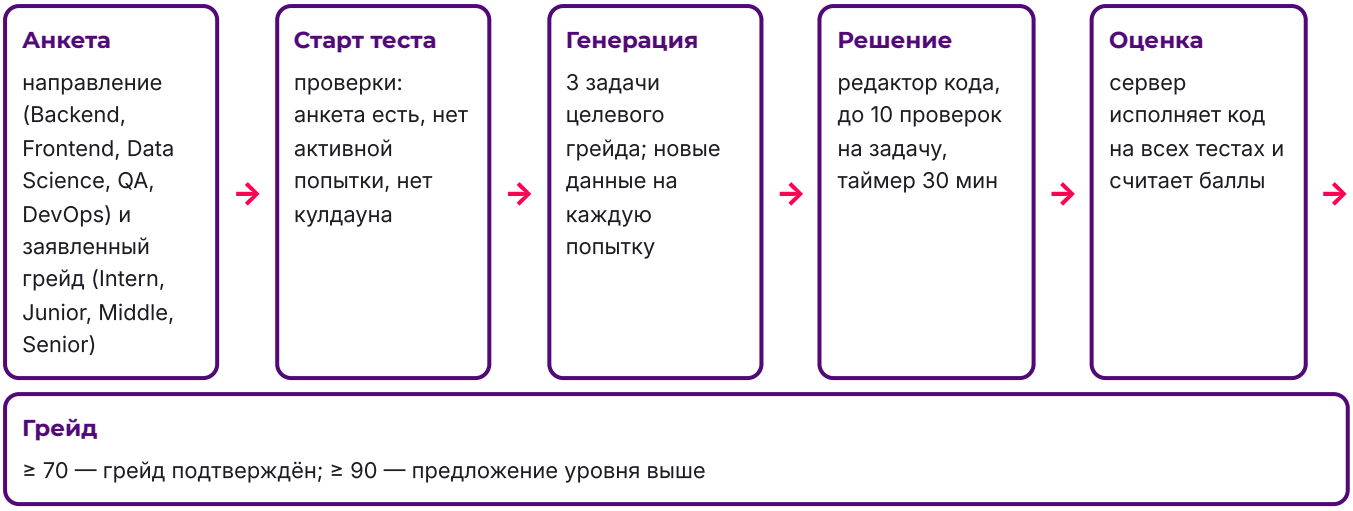
1. Работодатель вызывает `GET /api/candidates`; `matching_service` берёт пул видимых кандидатов (оба согласия, есть категория), `matching_engine` считает балл и причины; презентер скрывает контакты.

2. `POST /api/invitations`: проверяются заполненность профиля компании, способ связи, обязательная вилка ЗП и отсутствие активного приглашения той же паре; сохраняется снимок балла и причин; кандидату уходит письмо.
3. Кандидат открывает приглашение — статус автоматически становится *viewed*; принимает — *accepted*, в фоне отправляется подписанное событие в ATS работодателя.
4. При следующем открытии карточки у работодателя появляется основание `invitation_accepted`, контакты раскрываются, открытие карточки пишется в журнал доступа, который видит кандидат. Кандидат может закрыть доступ.

# 4 Механика тестирования и обоснование устойчивости

Тест — единственный источник подтверждённой категории. Он практический: кандидат пишет код на Python, который исполняется на сервере на скрытых тестах. Грейд присваивает бизнес-логика бэкенда по правилам ниже; LLM в тестировании не участвует.

## 4.1 Путь категоризации




## 4.2 Состав попытки и генерация заданий

- Каждая попытка содержит **ровно три задачи** целевого грейда в порядке сложности: лёгкая, средняя, сложная. Банк содержит 60 шаблонов (5 направлений × 4 грейда × 3 сложности, по одному на ячейку); разработку банка ведёт ML-участник команды.
- При старте сервер берёт криптографически случайный `seed` (`secrets.randbits(31)`), от него — отдельный генератор на каждую задачу. Шаблон функцией `make(rng)` порождает данные задачи: аргументы и ожидаемые результаты тестов.
- Полные задания с эталонами сохраняются только в попытке на сервере (`test_attempts.items`). Клиент получает урезанную версию: условие, сигнатуру функции, пример, баллы, счётчик проверок. Поля с ответами и тестами в схему ответа не входят — это проверяет автотест `test_answers_never_sent_to_frontend`.
- При старте можно указать вакансию — тогда навыки вакансии повышают вес подходящих шаблонов (механизм готов, эффект проявится при расширении банка).

## 4.3 Подсчёт баллов

| Задача | Базовые баллы × доля пройденных тестов | Бонус, если пройдены все тесты | Максимум |
|---|---|---|---|
| Лёгкая | 15 | 5 | 20 |
| Средняя | 20 | 10 | 30 |
| Сложная | 35 | 15 | 50 |
| **Итого** | | | **100** |

Баллы за задачу = `база × пройдено / всего + бонус (если пройдено всё)`. Например, сложная задача с 1 из 2 тестов даёт 17,5 балла, с 2 из 2 — 50. Итог округляется один раз до 0,1. Сервер дополнительно проверяет целостность матрицы баллов: попытка с повреждёнными весами не оценивается.

## 4.4 Предварительные проверки и таймер

- `POST /api/testing/attempts/{id}/check` прогоняет код на **полном** наборе тестов задачи и возвращает только «пройдено X из Y» и обобщённое сообщение — входные данные и ожидаемые значения скрытых тестов не раскрываются.
- Лимит — **10 проверок на задачу**; счётчик хранится в БД, строка попытки блокируется (`SELECT … FOR UPDATE`), поэтому лимит общий для всех вкладок и параллельных запросов. Синтаксические ошибки и запрещённые конструкции тоже расходуют проверку.
- Проверка не завершает попытку и не меняет грейд. Итог считается по коду, отправленному при сдаче.
- Длительность — 30 минут (плюс 1 минута на сетевые задержки при сдаче). Интерфейс сохраняет черновики и автоматически отправляет ответы по истечении времени. Просроченная и не сданная попытка помечается как истёкшая с сообщением «Время тестирования истекло» и на грейд не влияет.

## 4.5 Решение по грейду и ограничения частоты

| Ситуация | Правило |
|---|---|
| Балл ≥ 70 | Грейд подтверждён и присвоен; при смене грейда — запись в историю «Тест N/100» |
| Балл ≥ 90 и грейд ниже Senior | Предложение сразу пройти тест на уровень выше (исключение из 90-дневного кулдауна) |
| Балл < 70, грейд уже есть | Грейд сохраняется — **не понижается** |
| Балл < 70, грейда ещё нет | Сразу доступен тест другого уровня, в том числе ниже — попытка «не сжигается» (уточнение Q&A); каждый уровень — не чаще раза в 24 часа |
| Смена грейда | Не чаще раза в **90 дней** |
| Пересдача того же направления и грейда | Не чаще раза в **24 часа** (отсчёт от начала прошлой попытки) |
| Одновременно | Не больше одной активной попытки |

## 4.6 Безопасное исполнение кода кандидата

1. **Статический анализ AST:** длина до 10 000 символов; импорт только из `math, collections, itertools, functools, heapq, bisect, re, string, statistics`; запрещены `open, exec, eval, compile, __import__, getattr` и др.; запрещены «дандер»-атрибуты и пути к `sys/os/builtins` и кадрам стека (например, `statistics.sys`, `random._os`, `f_locals`); обязательно наличие функции из условия.
2. **Отдельный процесс:** `python -I` (изолированный режим), пустое окружение, таймаут 3 с, лимиты CPU 4 с и памяти 512 МБ.

3. **Разделение данных:** дочерний процесс получает только аргументы тестов. Ожидаемые ответы остаются в основном процессе, и сравнение выполняет он. Поэтому решение не может ни подсмотреть эталон, ни «напечатать» себе зачёт. Это закреплено автотестами `test_code_runner_security.py`: 9 попыток обхода и проверка работы честных решений.

## 4.7 Обоснование устойчивости к распространению заданий

Цель — чтобы знание чужих ответов не давало грейда. Устойчивость обеспечивается сочетанием мер, каждая из которых реализована в коде:

| Мера | Почему затрудняет «слив» ответов |
|---|---|
| Новые данные на каждую попытку | Криптографический seed; у 40 из 60 шаблонов меняются данные видимого примера. Скрытые проверки пока одинаковы между попытками (см. 4.8) — их рандомизация входит в план расширения банка |
| Практическая задача на код | Оценивается работающая функция, а не выбор варианта из списка: кандидат должен написать и отладить реализацию алгоритма за 30 минут |
| Скрытые тесты только на сервере | Тесты и эталоны не уходят в браузер; зачёт ставит сервер, подделать результат из браузера нельзя |
| Ограничение проверок | 10 проверок на задачу, результат — только число пройденных тестов: подобрать ответ перебором невозможно |
| Кулдауны | Пересдача того же уровня — раз в сутки, смена грейда — раз в 90 дней: нельзя «прокручивать» тест до удачного варианта |
| Таймер и одна активная попытка | 30 минут на три задачи; нельзя держать несколько попыток параллельно |
| Защищённое исполнение | Код не может прочитать эталон или подменить вердикт (п. 4.6) |
| Статистика по шаблонам | Таблица `question_stats` накапливает долю решений по каждому шаблону для последующего анализа аномально лёгких (утёкших) шаблонов (интерфейса анализа пока нет) |

Количественно: в синтетическом эксперименте (раздел 6) знание полного набора ответов *другого* варианта того же теста в среднем по грейдам даёт **45,6 из 100** баллов против 100 при полностью фиксированном тесте (Intern — 0, Junior и Middle — 50, Senior — 80). Метрика учитывает утечку ответов на данные, но не утечку кода решения: работающее решение проходит любой вариант, поэтому главная защита — необходимость самостоятельно написать работающий код за ограниченное время, лимит проверок и кулдауны.

## 4.8 Честные ограничения текущего банка

- В каждой ячейке «направление × грейд × сложность» сейчас один шаблон, а для одной ячейки «грейд × сложность» направления в основном используют один и тот же алгоритм с разной легендой: условия задач для пары «направление × грейд» постоянны, меняются данные.
- 20 из 60 шаблонов пока генерируют фиксированные данные; на них приходится 50 баллов у Junior и Middle и 80 у Senior.
- Первый тест задачи показывается как пример; скрытая часть — 1–2 теста, и она одинакова во всех попытках.
- Метод `GET /api/vacancies/{id}/assessment-preview` доступен любой роли и показывает условия задач категории до начала теста; в боевом режиме его нужно ограничить работодателем — владельцем вакансии.
- Прокторинга, антиплагиата и защиты от подсказок LLM нет (по Q&A это плюс, а не обязательное требование).

Расширение банка (несколько шаблонов на ячейку, рандомизация всех шаблонов, генерация скрытых тестов отдельно от примера) не требует изменений механики: движок уже выбирает шаблон случайно из пула ячейки и учитывает навыки вакансии. План — в разделе 14.

# 5 Механика подбора и логика категоризации

## 5.1 Категории

Категория — пара **направление · грейд** (например, «Бэкенд · Middle»): 5 направлений × 4 грейда = 20 категорий. Подтверждённая категория берётся только из завершённого теста. Кандидат без теста виден по заявленной категории с пометкой «Грейд заявлен кандидатом» и пониженным баллом. Работодатель видит сводку по категориям: число кандидатов, подтверждённых, неподтверждённых, с ФСП и средний балл теста (`GET /api/candidates/categories`).

## 5.2 Конвейер подборки

**1. Критерии** — из вакансии, сохранённой потребности или фильтров: направление, грейды (целевой ±1), стек, текст задачи

→

**2. Пул (SQL)** — оба согласия, активный аккаунт, есть категория; фильтр по направлению и грейдам

→

**3. Фильтры** — только с ФСП, только подтверждённые, мин. балл теста, формат работы, город

→

**4. Скоринг** — балл 0..100, причины и вклад факторов

→

**5. Выдача** — сортировка, страницы, блоки категорий; новая подборка не стирает сохранённые потребности

## 5.3 Формула соответствия

```
Оценка = 100 × ( Σ wᵢ·xᵢ / Σ wᵢ ) × M        (суммы — по факторам, для которых есть сигнал)

  x_test     = балл теста / 100 (0, если грейд не подтверждён)       w = 0.35
  x_skills   = доля требуемых навыков, которые есть у кандидата     w = 0.35
  x_fsp      = балл ФСП 0..1 (см. раздел 7.4)                        w = 0.15
  x_activity = 0.7·свежесть + 0.3·средняя оценка коротких заданий/10  w = 0.10
  x_text     = доля слов описания задачи, найденных в профиле/резюме  w = 0.05

  M = 0.7, если грейд отличается от целевого на одну ступень; 0.5 — на две и больше
  M × 0.6 дополнительно, если грейд не подтверждён тестом
```

- **Почему так.** Подтверждённые сигналы (тест и ФСП) дают половину веса; совпадение стека отвечает за соответствие конкретной потребности; активность и текст — уточняющие сигналы. Грейд кандидата гарантирован фильтром категории, а неподтверждённый грейд дополнительно опускается множителем 0.6, поэтому кандидат, подтвердивший грейд тестом, при прочих равных всегда выше.
- **Нормализация.** Если у запроса нет стека или текста, соответствующий фактор исключается, а веса остальных пересчитываются — балл остаётся сопоставимым.
- **Свежесть.** 1.0, если активность не старше 14 дней; далее линейно снижается до 0.2 к 180 дням.
- **Сортировка:** по баллу, затем по баллу теста, затем по id (детерминированный порядок).

## 5.4 Объяснимость выдачи

У каждого кандидата в выдаче — плашки причин и раскрываемый блок «Вклад в оценку соответствия» (значение, нормированный вес и вклад каждого фактора и множитель грейда). Примеры причин:

| Тип | Пример |
|---|---|
| test | «Бэкенд · Middle: тест 90/100» или «грейд заявлен кандидатом, тест ещё не пройден» |
| skills | «Стек совпадает 3 из 4: Python, PostgreSQL, Docker» + «Не хватает: Kafka» |
| fsp | «ФСП ID …: призёр 2 из 3 соревн., лучшее — 1 место · КМС» или «Нет истории ФСП — оценка по тесту» |
| strengths | «Сильные темы теста: …» (темы с результатом ≥ 80 %) |
| activity | «Активен 2 дн. назад · коротких заданий: 3 (ср. 8.3/10)» |
| text | «Опыт похож на задачи из описания» |
| grade | «Грейд Junior вместо Middle» (если грейд соседний с целевым) |

При отправке приглашения балл и причины сохраняются снимком в приглашении — кандидат видит, почему его пригласили.

## 5.5 Источники критериев

- **Вакансия** (`GET /api/vacancies/{id}/matches`): направление, грейд ±1, навыки, текст вакансии и команды; формат и город — для офисной работы.
- **Сохранённая потребность** (`GET /api/candidates?needs_id=`): направление, грейд ±1, стек, описание; дополнительные фильтры уточняют выдачу, не меняя потребность.
- **Свободный поиск** (`GET /api/candidates`, `POST /api/candidates/search`): направление, грейды, стек, текст, «только ФСП», «только подтверждённые», минимальный балл.

# 6 Процедура валидации и её результаты

Валидация состоит из пяти независимых проверок: (1) синтетическая проверка механики тестирования, (2) проверка качества подбора на размеченных парах «вакансия — кандидат», (3) проверка эталонными решениями всех 20 категорий, (4) автотесты и сквозной сценарий через API, (5) нагрузочная проверка. Все результаты ниже получены прогоном кода ветки main (коммит c19f7d3) 11.10.2026 и воспроизводятся командами из п. 6.6.

## 6.1 Методика синтетической валидации (`scripts/evaluate.py`)

Реальных кандидатов и эталонной разметки организаторы не предоставляют (Q&A), поэтому команда строит синтетическую популяцию с **известным истинным уровнем** и проверяет, восстанавливает ли его система.

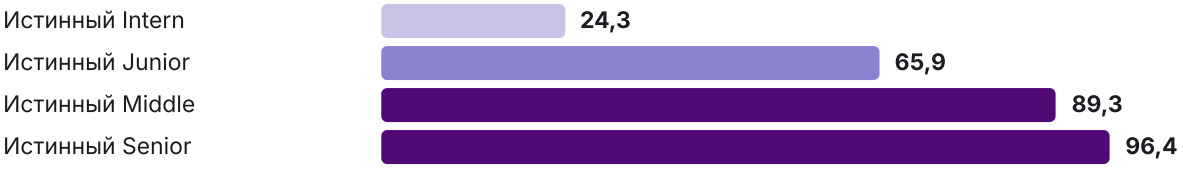
- **Популяция:** 400 кандидатов направления Backend, истинное умение θ равномерно на [0; 4), истинный грейд = ⌊θ⌋ (Intern…Senior). Генератор с фиксированным зерном — результат детерминирован.
- **Модель ответа** (IRT 2PL): вероятность решить задачу уровня L равна `1 / (1 + e^(−1.7·(θ − (L − 0.75))))`.
- **Путь кандидата** повторяет правила продукта: заявленный грейд с ошибкой ±1 у 40 %; реальный генератор вариантов `build_items` (3 задачи, 20/30/50); порог 70, при ≥ 90 — попытка уровнем выше; провал без грейда — шаг вниз.
- **Подбор:** 900 синтетических кандидатов, 40 вакансий; кандидат релевантен вакансии, если совпадает направление, истинный грейд равен грейду вакансии и истинный стек покрывает ≥ 60 % требований. Сравниваются реальный ранжировщик `matching_engine.rank`, поиск по ключевым словам резюме и сортировка только по баллу теста.

## 6.2 Результаты: тестирование и категоризация

| Метрика (критерий 2 ТЗ) | Значение | Что означает |
|---|---|---|
| Точность грейда (совпадение с истинным) | **0.61** | 63,2 % для кандидатов не на границе уровней |
| Точность грейда ±1 ступень | **0.953** | грубых ошибок категоризации меньше 5 % |
| Согласованность при повторном прохождении (test-retest) | **0.565** / ±1: **0.907** | повтор даёт тот же или соседний грейд в 90,7 % случаев |
| Дискриминативность заданий (точечно-бисериальная корреляция) | средняя **0.751**, минимальная **0.664** | в рамках принятой IRT-модели (параметры заданы, а не оценены на реальных данных) каждое задание разделяет слабых и сильных (ориентир > 0.3) |
| Доля кандидатов без грейда | 0.045 | не прошли даже уровень Intern |

| Метрика (критерий 2 ТЗ) | Значение | Что означает |
|---|---|---|
| Балл за ответы чужого варианта («слив» ответов на данные) | **45,6** из 100 в среднем (фиксированный тест — 100) | Intern 0, Junior/Middle 50, Senior 80; утечка кода решения не моделируется (см. 4.7) |

**Разделяющая сила теста уровня Middle** — средний балл по истинным грейдам монотонно растёт:




Порог прохождения 70 отделяет Middle и Senior (выше) от Junior и Intern (ниже). Значения получены на синтетической модели ответа, а не на реальных решениях.

## 6.3 Результаты: качество подбора (критерий 1 ТЗ)

| Ранжировщик | P@10 | nDCG@10 | Доля верного грейда в топ-10 |
|---|---|---|---|
| **Наш (matching_engine)** | **0.365** | **0.457** | **0.76** |
| Поиск по ключевым словам резюме | 0.185 | 0.191 | 0.28 |
| Только балл теста | 0.208 | 0.209 | 0.755 |




Наш подбор вдвое точнее поиска по ключевым словам (P@10 0.365 против 0.185; nDCG@10 0.457 против 0.191) и в 2,7 раза чаще ставит в топ кандидатов верного грейда (0.76 против 0.28; базовая линия не фильтрует по грейду, поэтому этот выигрыш в основном даёт категорийный фильтр). Выигрыш над сортировкой только по тесту показывает вклад совпадения стека и ФСП.

## 6.4 Эталонные решения, автотесты и сквозной сценарий

- **Эталонные решения** (`scripts/verify_hybrid_testing.py`): для всех 20 пар «направление × грейд» тест содержит 3 задачи нужного уровня в порядке сложности с весами 20/30/50, эталонные решения проходят через настоящий исполнитель кода и получают **100/100**, публичная часть заданий не содержит тестов. Итог: **OK: contest-scoring-all-5**.
- **Автотесты: 182 теста** в 17 файлах — **182 passed** на SQLite и **182 passed** на PostgreSQL 16. Покрывают авторизацию и роли, ротацию токенов, лимиты, шифрование, тестирование и баллы, лимит проверок, попытки взлома исполнителя кода, категоризацию и кулдауны, подбор и ФСП, приглашения и контакты, вакансии, отклики, задания, ATS, контракты ML и фронтенда.
- **CI (GitHub Actions)** на каждый push и pull request, затрагивающий `backend/` или `frontend/`: тесты на SQLite и PostgreSQL, миграции вверх — вниз — вверх, генерация демо-данных, проверка шифрования; отдельно — `npm ci`, lint и сборка фронтенда.
- **Сквозной сценарий через API** на запущенном сервере с PostgreSQL — **20 из 20** проверок пройдены: регистрация и подтверждение почты; вход до подтверждения запрещён; тест без анкеты запрещён; 3 задачи 20/30/50; ответы не уходят клиенту; счётчик проверок; провал не присваивает грейд, уровень ниже доступен сразу; кандидат виден как «не подтверждён» без контактов; приглашение без зарплаты отклоняется; статус «просмотрено»; принятие открывает

контакты; запись в журнале; отзыв доступа снова скрывает контакты; другой работодатель контактов не видит.

## 6.5 Нагрузочная проверка (ТЗ, п. 3.5: «десятки пользователей»)

`scripts/load_test.py`: 50 одновременных виртуальных пользователей (на 20 учётных записях, запросы напрямую в backend :8000) в течение 30 секунд выполняют типичные операции. Стенд: PostgreSQL 16, uvicorn 4 процесса, демо-база (200 кандидатов), 2 vCPU; генератор нагрузки работал на той же машине.

| Операция | Запросов | Медиана, мс | 95 %, мс | 99 %, мс |
|---|---|---|---|---|
| Подборка под вакансию | 474 | 338 | 1521 | 2652 |
| Поиск по категории | 474 | 357 | 1645 | 3024 |
| Открыть профиль кандидата | 474 | 285 | 1629 | 2277 |
| Свой профиль | 464 | 322 | 1688 | 2517 |
| Мои приглашения | 464 | 338 | 1525 | 2623 |
| Список вакансий | 464 | 405 | 1564 | 2515 |

Всего **2814 запросов** (91 в секунду), сервер не вернул ни одной ошибки 4xx/5xx; 4 запроса (0,14 %) завершились сетевой ошибкой на стороне генератора нагрузки. Медианное время основных операций — 0,3–0,4 с на 2 vCPU. На стенде жюри (около 8 CPU) рекомендуется `WEB_CONCURRENCY=8`.

## 6.6 Как воспроизвести

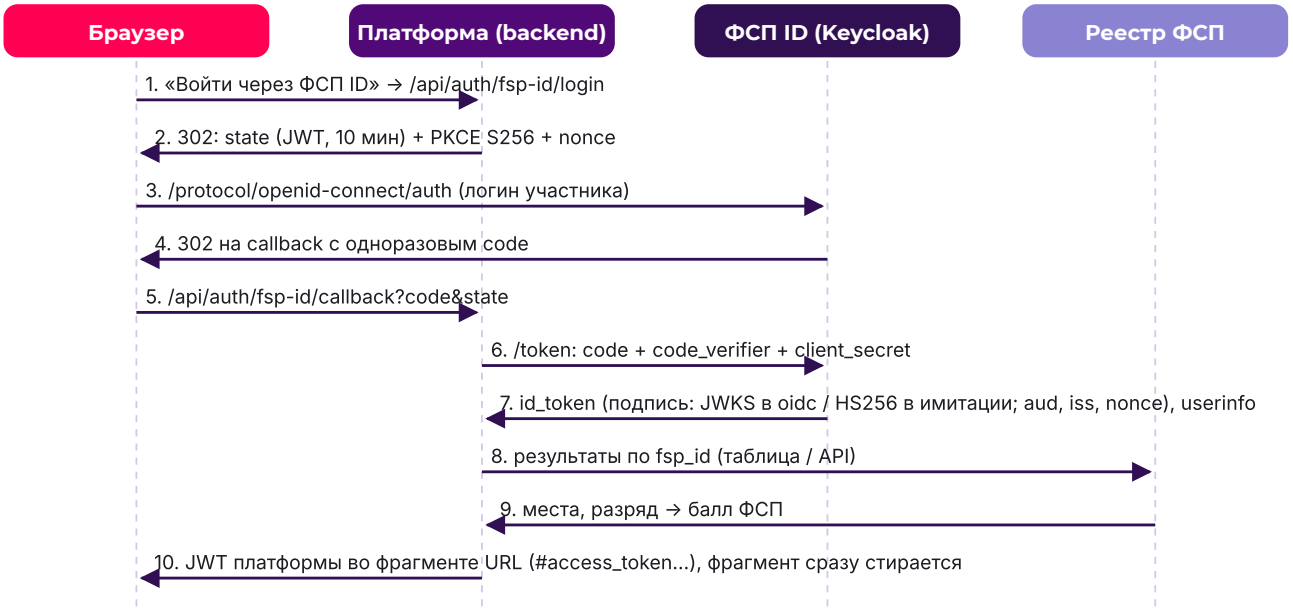
```
# из папки backend (Python 3.12)
pip install -r requirements-dev.txt
python -m pytest -q                                  # 182 теста на SQLite
TEST_DATABASE_URL=postgresql+psycopg://user:pass@localhost/test_db python -m pytest -q   #
только пустая тестовая БД: тесты пересоздают таблицы
python -m scripts.verify_hybrid_testing              # эталонные решения 20 категорий
python -m scripts.evaluate                           # метрики 6.2–6.3, отчёт в scripts/output/
# нагрузка — на запущенном стенде с демо-данными
python -m scripts.load_test --users 50 --seconds 30 --url http://localhost:8000
```

> **Ограничения валидации.** Синтетическая модель считает каждую задачу решённой целиком или нет (частичные баллы не моделируются), не исполняет код и рассчитана для направления Backend. Метрики подбора получены на синтетических парах. Жюри по регламенту проверяет решение на собственном закрытом наборе; процедура и скрипты позволяют повторить расчёт на любых размеченных данных.

# 7 Схема интеграции с реестром ФСП и ФСП ID

Боевого доступа к ФСП ID (Keycloak) и API реестра на хакатоне нет, поэтому интеграция реализована «скелетом» на стандартных протоколах с имитацией провайдера — как и рекомендовано в Q&A. Переход на боевой контур — изменение настроек, а не кода.

## 7.1 Вход через ФСП ID (OpenID Connect, Authorization Code + PKCE)




- Совместимость с Keycloak: endpoints реалма (`/.well-known/openid-configuration`, `/protocol/openid-connect/auth|token|userinfo|certs`), в JWT платформы роль передаётся в claim `realm_access.roles` — как у Keycloak.
- Поиск аккаунта: по `sub` → по подтверждённому email (привязка) → создание нового кандидата с подтверждённой почтой. Токены ФСП ID не сохраняются — только `sub`.
- Если в профиле ФСП ID есть claim `fsp_id`, достижения подтягиваются из реестра автоматически.
- Режимы `FSP_ID_MODE`: `mock` — встроенная имитация реалма (демо-учётки, пароль `fsp12345`); `oidc` — боевой провайдер по `FSP_ID_ISSUER` (подпись RS256/ES256 через JWKS); `off` — кнопка скрыта.

## 7.2 Что нужно для подключения боевого ФСП ID

1. Зарегистрировать в реалме ФСП confidential-клиента, redirect URI `https://<домен>/api/auth/fsp-id/callback`, scopes `openid email profile`.
2. Добавить protocol mapper, выдающий номер участника в claim `fsp_id` (в userinfo или id_token).
3. На платформе задать `FSP_ID_MODE=oidc`, `FSP_ID_ISSUER`, `FSP_ID_CLIENT_ID`, `FSP_ID_CLIENT_SECRET`, `BACKEND_PUBLIC_URL`. В режиме prod сервер не стартует без issuer и с dev-секретом.

## 7.3 Реестр результатов: контракт данных

Реестр составной: сначала таблица реальных результатов `backend/data/fsp_results.xlsx` (или csv; файл перечитывается при изменении, папка смонтирована в контейнер без пересборки), затем — при `FSP_DEMO_FALLBACK=true` — демо-реестр. Шаблон с инструкцией и выпадающими списками:

`backend/data/fsp_results_template.xlsx`. Команды: `python -m scripts.fsp_data template | check | sync` — создать шаблон, проверить таблицу, обновить достижения у уже привязанных кандидатов.

| Колонка | Смысл и правило проверки |
|---|---|
| fsp_id | номер участника, обязательно; одинаков у всех строк одного человека |
| event, discipline | соревнование (пусто — участник без достижений), дисциплина |
| level | международный / всероссийский / региональный / муниципальный |
| year | год от 2015 до текущего |
| place, result | 1, 2, 3 или пусто; текстовый результат («финалист», «участник») |
| team, sport_rank | команда; спортивный разряд (от 3 юношеского до ЗМС) |
| source_url, comment | ссылка на протокол (http/https), комментарий |

Достижения из таблицы помечаются как подтверждённые (`verified`), из демо-реестра — как `demo`; статус виден работодателю в карточке. Для будущего API реестра предлагается тот же контракт: `GET /participants/{fsp_id}/results` → список записей с полями таблицы; клиент подключается вместо загрузчика таблицы без изменения остальной логики.

## 7.4 Балл ФСП

```
очки = Σ за призовые места (1 место — 1.0, 2 — 0.85, 3 — 0.7)
     + min(0.6, Σ за непризовые: финал — 0.3, участие — 0.15)
балл ФСП = 1 − e^(−очки / 1.2)        одна победа ≈ 0.57, два призовых места ≈ 0.79
```

Учитывается результативность, а не число явок: сколько угодно участий без призов дают не больше 0.39. Уровень соревнования показывается, но на балл не влияет — все соревнования равно важны (рекомендация Q&A). Без достижений балл 0, кандидат остаётся в выдаче. ФСП влияет на ранжирование внутри категории (вес 0.15) и **никогда не меняет грейд**.

# 8 Описание API

REST API на FastAPI, все методы описаны спецификацией **OpenAPI 3.1** и доступны интерактивно: Swagger UI — `/docs`, ReDoc — `/redoc`, JSON — `/openapi.json` (на локальном стенде — `http://localhost:8000/docs`). Итого **84 операции на 71 пути в 13 разделах**, 81 схема данных; дополнительно 2 служебных маршрута скрыты из схемы (страница подтверждения email по ссылке из письма и отправка формы входа имитации ФСП ID).

## 8.1 Общие правила

- Префикс `/api`, формат JSON (UTF-8); загрузка резюме — `multipart/form-data`.
- Аутентификация: заголовок `Authorization: Bearer <access_token>`. Access-токен живёт 15 минут, refresh — 14 дней с ротацией: повторное использование старого refresh-токена отзывает всю цепочку сессии (`TOKEN_REUSED`). В Swagger кнопка Authorize работает через `/api/auth/token` (username = email).
- Роль проверяется на сервере зависимостями: «кандидат», «работодатель», «любая роль». Чужие объекты отвечают 404, чтобы не раскрывать факт их существования.
- Единый формат ошибки: `{"code": "MACHINE_CODE", "message": "текст для пользователя", "detail": "…", "fields": {…}}`. Ошибки валидации — 422 `VALIDATION_ERROR` с сообщениями по полям. 429 содержит `retry_after` и заголовок `Retry-After`.
- Коды: 400 — некорректный запрос; 401 — нет или неверный токен; 403 — недостаточно прав или не подтверждена почта; 404 — не найдено; 409 — конфликт с текущим состоянием (статусы, кулдауны, дубликаты); 422 — ошибка полей; 429 — слишком много запросов.
- Пагинация: `page` (с 1) и `size` / `page_size` (1..100, по умолчанию 20); ответ подборки — `{total, page, size, categories, items}`, ленты вакансий — `{total, items}`.
- Служебный `GET /api/health`: статус, версия `1.4.0`, отпечаток кода и номер миграции — видно, какая именно сборка запущена.

## 8.2 Полный перечень методов

Таблица сформирована автоматически из спецификации OpenAPI работающего приложения; колонка «Доступ» — из реальных зависимостей авторизации маршрутов. Перечень ошибок взят из OpenAPI и может быть неполным: например, `POST /api/invitations` возвращает также 400 (`COMPANY_PROFILE_INCOMPLETE`, `CONTACT_REQUIRED`), защищённые методы — 401.

### Авторизация 11

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| POST | /api/auth/register | Регистрация кандидата или работодателя | без входа | RegisterRequest | 201 RegisterResponse | 409, 422, 429 |
| POST | /api/auth/verify-email | Подтвердить email (токен из письма) | без входа | VerifyEmailRequest | 200 UserOut | 401, 422 |
| POST | /api/auth/resend-verification | Отправить письмо подтверждения ещё раз | без входа | ResendVerificationRequest | 200 Message | 422 |
| POST | /api/auth/login | Вход (JSON) — для фронтенда | без входа | LoginRequest | 200 TokenPair | 401, 403, 422, 429 |

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| POST | /api/auth/token | Вход через форму — для кнопки Authorize в Swagger (username = email) | без входа | форма | 200 TokenPair | 401, 403, 422, 429 |
| POST | /api/auth/refresh | Обновить пару токенов (ротация) | без входа | RefreshRequest | 200 TokenPair | 401, 422 |
| POST | /api/auth/logout | Выход (отзыв сессии) | любая роль | Data | 204 | 422 |
| GET | /api/auth/me | Кто я + чек-лист онбординга | любая роль | — | 200 MeOut | 401 |
| GET | /api/auth/fsp-id/config | Показывать ли кнопку «Войти через FSP ID» | без входа | — | 200 FspIdConfig | |
| GET | /api/auth/fsp-id/login | Начать вход через FSP ID (переадресация на страницу FSP ID) | без входа | — | 302 | 422 |
| GET | /api/auth/fsp-id/callback | Сюда FSP ID возвращает пользователя после входа | без входа | — | 200 FspIdLoginResult, 302 | 422 |

## Справочники 1

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| GET | /api/reference | Все справочники одним запросом (кэшируйте на фронте) | без входа | — | 200 | |

## Кандидат: профиль 13

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| GET | /api/candidate/profile | Мой профиль | кандидат | — | 200 CandidateProfileOut | 401, 403 |
| PATCH | /api/candidate/profile | Обновить профиль (только присланные поля) | кандидат | CandidateProfileUpdate | 200 CandidateProfileOut | 401, 403, 422 |
| POST | /api/candidate/resume/parse | Загрузить PDF-резюме и распознать поля | кандидат | форма | 200 ResumeParseOut | 400, 401, 403, 422 |
| GET | /api/candidate/profile/pdf | Скачать стандартизированный PDF-профиль | кандидат | — | 200 | 401, 403 |
| POST | /api/candidate/survey | Шаг 1: анкета — IT-направление и грейд, на который претендую | кандидат | SurveyIn | 200 CandidateProfileOut | 401, 403, 422 |
| GET | /api/candidate/category | Моя категория, грейд, когда можно менять, на какие грейды можно пройти тест | кандидат | — | 200 CategoryStatus | 401, 403 |
| PUT | /api/candidate/consents | Согласия (152-ФЗ) | кандидат | Consents | 200 CandidateProfileOut | 400, 401, 403, 422 |

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| PUT | /api/candidate/privacy | Что видно работодателю | кандидат | Privacy | 200 CandidateProfileOut | 401, 403, 422 |
| POST | /api/candidate/fsp | Привязать ФСП ID и подтянуть достижения из реестра | кандидат | FspLinkIn | 200 CandidateProfileOut | 401, 403, 404, 409, 422 |
| DELETE | /api/candidate/fsp | Отвязать ФСП ID | кандидат | — | 200 CandidateProfileOut | 401, 403 |
| GET | /api/candidate/contact-access-log | Кто и когда получил доступ к моим контактам | кандидат | — | 200 | 401, 403 |
| GET | /api/candidate/export | Выгрузить все мои данные (JSON) | кандидат | — | 200 | 401, 403 |
| DELETE | /api/candidate/account | Удалить аккаунт и все мои данные | кандидат | DeleteAccountIn | 204 | 401, 403, 422 |

## Кандидат: тестирование 6

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| GET | /api/testing/attempts | История моих попыток | кандидат | — | 200 list[AttemptSummary] | 401, 403 |
| POST | /api/testing/attempts | Шаг 2–3: выбрать грейд и начать тест | кандидат | StartAttemptIn | 201 AttemptOut | 401, 403, 404, 409, 422 |
| GET | /api/testing/attempts/{attempt_id} | Получить попытку (например, после перезагрузки страницы) | кандидат | — | 200 AttemptOut | 401, 403, 404, 422 |
| POST | /api/testing/attempts/{attempt_id}/check | Проверить код на тестах — до 10 раз на задание, без завершения попытки | кандидат | CodeCheckIn | 200 CodeCheckOut | 400, 401, 403, 404, 409, 422 |
| POST | /api/testing/attempts/{attempt_id}/submit | Отправить ответы и получить результат + решение по грейду | кандидат | SubmitIn | 200 AttemptResultOut | 400, 401, 403, 404, 409, 422 |
| GET | /api/testing/attempts/{attempt_id}/result | Результат завершённой попытки | кандидат | — | 200 AttemptResultOut | 401, 403, 404, 409, 422 |

## Работодатель: подбор 14

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| GET | /api/employer/profile | Профиль компании | работодатель | — | 200 EmployerProfileOut | 401, 403 |
| PATCH | /api/employer/profile | Обновить профиль компании | работодатель | EmployerProfileUpdate | 200 EmployerProfileOut | 401, 403, 422 |
| GET | /api/employer/needs | Мои сохранённые потребности («кого ищем») | работодатель | — | 200 list[NeedOut] | 401, 403 |

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| POST | /api/employer/needs | Сохранить новую потребность | работодатель | NeedIn | 201 NeedOut | 401, 403, 422 |
| GET | /api/employer/needs/{need_id} | Одна потребность | работодатель | — | 200 NeedOut | 401, 403, 404, 422 |
| PUT | /api/employer/needs/{need_id} | Изменить потребность (присылается целиком) | работодатель | NeedIn | 200 NeedOut | 401, 403, 404, 422 |
| DELETE | /api/employer/needs/{need_id} | Удалить потребность | работодатель | — | 204 | 401, 403, 404, 422 |
| GET | /api/candidates | Подборка через параметры адреса (удобно для фильтров на фронте) | работодатель | — | 200 SearchOut | 401, 403, 422 |
| GET | /api/candidates/categories | Обзор банка: категории и сколько в них кандидатов | работодатель | — | 200 list[CategoryBucket] | 401, 403 |
| POST | /api/candidates/search | Подборка: фильтры + ранжирование + обоснование каждого кандидата | работодатель | SearchIn | 200 SearchOut | 401, 403, 422 |
| GET | /api/candidates/{candidate_id} | Карточка кандидата (контакты — только после принятия инвайта/отклика) | работодатель | — | 200 CandidateDetail | 401, 403, 404, 422 |
| GET | /api/shortlist | Избранные кандидаты | работодатель | — | 200 list[ShortlistOut] | 401, 403 |
| POST | /api/shortlist | Добавить в избранное | работодатель | ShortlistIn | 201 ShortlistOut | 401, 403, 404, 409, 422 |
| DELETE | /api/shortlist/{candidate_id} | Убрать из избранного | работодатель | — | 204 | 401, 403, 404, 422 |

## Работодатель: интеграция с ATS 4

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| GET | /api/employer/integrations/ats | Настройки отправки кандидатов в вашу ATS и последние отправки | работодатель | — | 200 AtsConfigOut | 401, 403 |
| PUT | /api/employer/integrations/ats | Подключить ATS: адрес вебхука (секрет выдаётся один раз) | работодатель | AtsConfigIn | 200 AtsConfigOut | 400, 401, 403, 422 |
| DELETE | /api/employer/integrations/ats | Отключить интеграцию с ATS | работодатель | — | 204 | 401, 403 |
| POST | /api/employer/integrations/ats/test | Отправить в ATS тестовое событие | работодатель | — | 200 AtsDeliveryOut | 401, 403, 404 |

## Приглашения (главная механика) 8

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| POST | /api/invitations | Работодатель: пригласить кандидата (вилка ЗП обязательна) | работодатель | InvitationCreate | 201 InvitationOut | 401, 403, 404, 409, 422 |
| GET | /api/invitations | Мои приглашения: работодатель — отправленные, кандидат — входящие | любая роль | — | 200 list[InvitationOut] | 401, 403, 422 |
| GET | /api/invitations/{invitation_id} | Открыть приглашение (кандидату — автоматически статус viewed) | любая роль | — | 200 InvitationOut | 401, 403, 404, 422 |
| POST | /api/invitations/{invitation_id}/accept | Кандидат: принять — контакты станут видны работодателю | кандидат | Data | 200 InvitationOut | 401, 403, 404, 409, 422 |
| POST | /api/invitations/{invitation_id}/decline | Кандидат: отклонить | кандидат | Data | 200 InvitationOut | 401, 403, 404, 409, 422 |
| PATCH | /api/invitations/{invitation_id}/answer | Кандидат: ответить одним запросом (status = accepted \| rejected) | кандидат | InvitationAnswer | 200 InvitationOut | 401, 403, 404, 409, 422 |
| POST | /api/invitations/{invitation_id}/withdraw | Работодатель: отозвать приглашение | работодатель | — | 200 InvitationOut | 401, 403, 404, 409, 422 |
| PUT | /api/invitations/{invitation_id}/contact-access | Кандидат: закрыть свои контакты от компании (или открыть снова) после принятия | кандидат | ContactAccessIn | 200 InvitationOut | 401, 403, 404, 409, 422 |

## Вакансии и отклики 11

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| GET | /api/vacancies | Лента вакансий (для кандидата и всех) | любая роль | — | 200 Page_VacancyOut_ | 401, 403, 422 |
| POST | /api/vacancies | Работодатель: опубликовать вакансию (= описать потребность) | работодатель | VacancyCreate | 201 VacancyOut | 401, 403, 422 |
| GET | /api/vacancies/mine | Работодатель: мои вакансии | работодатель | — | 200 list[VacancyOut] | 401, 403 |
| GET | /api/vacancies/{vacancy_id} | Вакансия | любая роль | — | 200 VacancyOut | 401, 403, 404, 422 |
| PATCH | /api/vacancies/{vacancy_id} | Работодатель: редактировать / закрыть (status=closed) | работодатель | VacancyUpdate | 200 VacancyOut | 400, 401, 403, 404, 422 |
| GET | /api/vacancies/{vacancy_id}/matches | Работодатель: подборка кандидатов под вакансию с обоснованием | работодатель | — | 200 SearchOut | 401, 403, 404, 422 |

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| GET | /api/vacancies/{vacancy_id}/assessment-preview | Как система сформирует тест под эту вакансию (2 разных варианта) | любая роль | — | 200 AssessmentPreviewOut | 401, 403, 404, 422 |
| POST | /api/vacancies/{vacancy_id}/apply | Кандидат: откликнуться самому | кандидат | Data | 201 ApplicationOut | 400, 401, 403, 404, 409, 422 |
| GET | /api/applications | Отклики: кандидат — мои, работодатель — на мои вакансии (?vacancy_id=) | любая роль | — | 200 list[ApplicationOut] | 401, 403, 422 |
| PATCH | /api/applications/{application_id} | Работодатель: просмотрено / принять / отклонить отклик | работодатель | ApplicationStatusUpdate | 200 ApplicationOut | 401, 403, 404, 409, 422 |
| POST | /api/applications/{application_id}/withdraw | Кандидат: отозвать отклик | кандидат | — | 200 ApplicationOut | 401, 403, 404, 409, 422 |

## Короткие задания 7

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| POST | /api/short-tasks | Работодатель: создать задание | работодатель | ShortTaskCreate | 201 ShortTaskOut | 401, 403, 422 |
| GET | /api/short-tasks/mine | Работодатель: мои задания | работодатель | — | 200 list[ShortTaskOut] | 401, 403 |
| POST | /api/short-tasks/{task_id}/close | Работодатель: закрыть | работодатель | — | 200 ShortTaskOut | 401, 403, 404, 422 |
| GET | /api/short-tasks/feed | Кандидат: задания для моей категории | кандидат | — | 200 list[ShortTaskOut] | 401, 403 |
| POST | /api/short-tasks/{task_id}/submit | Кандидат: отправить решение или подход | кандидат | SubmissionCreate | 201 SubmissionOut | 400, 401, 403, 404, 409, 422 |
| GET | /api/short-tasks/{task_id}/submissions | Работодатель: решения по заданию | работодатель | — | 200 list[SubmissionOut] | 401, 403, 404, 422 |
| POST | /api/short-tasks/submissions/{submission_id}/review | Работодатель: оценить решение (0..10) | работодатель | SubmissionReview | 200 SubmissionOut | 401, 403, 404, 422 |

## ФСП (реестр достижений) 1

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| GET | /api/fsp/registry/{fsp_id} | Что знает реестр ФСП об участнике: реальные результаты из таблицы или демо-данные | без входа | — | 200 | 404, 422 |

## FSP ID (имитация Keycloak) 5

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| GET | `/api/mock-fsp-id/realms/fsp/.well-known/openid-configuration` | Discovery: адреса провайдера (как у Keycloak) | без входа | — | 200 | |
| GET | `/api/mock-fsp-id/realms/fsp/protocol/openid-connect/certs` | Ключи провайдера (в имитации id_token подписан HS256 секретом клиента) | без входа | — | 200 | |
| GET | `/api/mock-fsp-id/realms/fsp/protocol/openid-connect/auth` | Страница входа FSP ID (имитация) | без входа | — | 200 | |
| POST | `/api/mock-fsp-id/realms/fsp/protocol/openid-connect/token` | Обмен кода на токены (имитация) | без входа | форма | 200 | 422 |
| GET | `/api/mock-fsp-id/realms/fsp/protocol/openid-connect/userinfo` | Профиль участника FSP ID (имитация) | без входа | — | 200 | |

## Тестовый приёмник ATS (имитация) 2

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| POST | `/api/mock-ats/webhook` | Принять событие платформы, как это сделала бы ATS (проверка подписи) | без входа | — | 200 MockAtsReceipt | 400, 401, 404 |
| GET | `/api/mock-ats/events` | Работодатель: что получил тестовый приёмник (последние 20 событий) | работодатель | — | 200 list[MockAtsEventOut] | 401, 403, 404 |

## Служебное 1

| Метод | Путь | Назначение | Доступ | Тело запроса | Успешный ответ | Ошибки |
|---|---|---|---|---|---|---|
| GET | `/api/health` | Проверка, что сервис жив и какой код запущен | без входа | — | 200 | |

# 9 Библиотеки и компоненты с версиями

Все библиотеки и компоненты стека — открытое ПО под свободными лицензиями (MIT, BSD, Apache-2.0, PostgreSQL License, LGPL-3.0 у psycopg и аналогичные); GigaChat — внешний облачный сервис, его использование необязательно. Версии бэкенда закреплены в `requirements.txt`, фронтенда — в `package-lock.json`.

## 9.1 Бэкенд (Python 3.12)

| Пакет | Версия | Назначение |
|---|---|---|
| fastapi | 0.115.12 | веб-фреймворк, OpenAPI/Swagger |
| uvicorn[standard] | 0.34.2 | ASGI-сервер |
| sqlalchemy | 2.0.41 | ORM |
| alembic | 1.16.1 | миграции схемы БД |
| psycopg[binary] | 3.2.9 | драйвер PostgreSQL |
| pydantic / pydantic-settings | 2.11.5 / 2.9.1 | схемы и валидация; настройки из окружения |
| email-validator | 2.2.0 | проверка адресов email |
| PyJWT | 2.10.1 | JWT, проверка id_token ФСП ID |
| python-multipart | 0.0.20 | загрузка файлов и формы |
| httpx | 0.28.1 | HTTP-клиент (ATS, OIDC, внешний ML) |
| cryptography | 45.0.5 | AES-256-GCM для персональных данных |
| pypdf | 5.6.0 | извлечение текста из PDF-резюме |
| reportlab | 4.4.1 | генерация PDF-профиля |
| openpyxl | 3.1.5 | таблица результатов ФСП |
| gigachat | 0.2.3 | клиент GigaChat (разбор резюме) |
| Faker | 37.3.0 | генерация демо-данных |
| pytest (dev) | 8.3.5 | автотесты |

## 9.2 Фронтенд

| Пакет | Версия (package-lock) | Назначение |
|---|---|---|
| react / react-dom | 19.3.0 | интерфейс |
| vite | 8.3.3 | сборка и dev-сервер с прокси `/api` |
| tailwindcss / @tailwindcss/vite | 4.3.3 | стили, фирменная палитра ФСП |
| monaco-editor / @monaco-editor/react | 0.57.0 / 4.7.0 | редактор кода в тесте |
| @vitejs/plugin-react | 6.1.2 | поддержка React в Vite |
| eslint, @eslint/js, eslint-plugin-react-hooks | 10.12.0, 10.0.1, 7.1.1 | статический анализ |

## 9.3 Инфраструктура

| Компонент | Версия | Назначение |
|---|---|---|
| PostgreSQL | 16 ( `postgres:16-alpine` ) | СУБД |
| Python | 3.12 ( `python:3.12-slim` ) | среда бэкенда |
| Node.js | 22 ( `node:22-alpine` ) | сборка фронтенда |
| nginx | 1.27 ( `nginx:1.27-alpine` ) | раздача сайта, прокси API |
| Mailpit | `axllent/mailpit:latest` | тестовая почта (для боевого контура — внешний SMTP) |
| Docker Compose | v2 | оркестрация контейнеров |
| GigaChat API | модель GigaChat-Pro (настраивается `GIGA_MODEL` ) | необязательный облачный сервис для разбора резюме |

# 10 Сборка, развёртывание и локальный запуск

> **Готовый стенд для проверки:** https://lct.demo.bloomee.ru · письма подтверждения: https://mail.lct.demo.bloomee.ru · демо-вход — кнопками на экране входа (пароль `demo12345`).

## 10.1 Требования

- Git и Docker с Docker Compose v2 (Docker Desktop на Windows/macOS или Docker Engine с плагином compose на Linux). ОС не важна.
- Ресурсы: демо-стенд проверен под нагрузкой 50 пользователей на 2 vCPU (раздел 6.5); GPU не нужен. Свободные порты 5173, 8000, 8025.
- Ключ GigaChat не обязателен — без него работает встроенный парсер резюме.

## 10.2 Запуск за пять команд

```
git clone https://github.com/NedYouNet/LLH_BY_KARMA.git
cd LLH_BY_KARMA
git switch main
docker compose up -d --build                          # первая сборка — несколько минут
docker compose exec backend python -m scripts.seed    # демо-данные (один раз)
```

Миграции БД применяются автоматически при старте бэкенда. Команду seed стоит запускать через 15–20 секунд после старта. Повторный запуск без флага безопасен («Демо-данные уже есть»); `--reset` полностью пересоздаёт демо-базу и **удаляет все данные**, включая созданные вручную аккаунты.

| Что открыть | Адрес |
|---|---|
| Сайт | http://localhost:5173 |
| Swagger / ReDoc | http://localhost:8000/docs · http://localhost:8000/redoc |
| Версия сборки | http://localhost:8000/api/health → `version 1.4.0`, `migration 0006` |
| Письма (Mailpit) | http://localhost:8025 |

## 10.3 Демо-аккаунты (пароль у всех `demo12345`)

| Роль | Email | Состояние |
|---|---|---|
| Кандидат | candidate@demo.ru | Бэкенд · Middle, тест пройден, есть ФСП, входящее приглашение |
| Новый кандидат | newbie@demo.ru | только зарегистрирован: анкета и тест с нуля |
| Работодатель | employer@demo.ru | «ООО Цифровые Решения», вакансии, приглашения, отклики |
| Ещё работодатели и кандидаты | hr1…hr5@demo.ru, cand1… cand199@demo.ru | наполнение базы |
| Вход через ФСП ID (имитация) | candidate@demo.ru, maria.fsp@demo.ru и др. | пароль `fsp12345` на странице имитации |

## 10.4 Управление стендом

```
docker compose ps                          # состояние контейнеров
docker compose logs backend --tail 100     # журнал бэкенда
docker compose stop / start                # остановить / запустить, база сохраняется
docker compose up -d --build               # после обновления исходников
docker compose down                        # удалить контейнеры (том базы остаётся)
docker compose down -v                     # удалить и базу
```

## 10.5 Необязательные настройки (файл `.env` в корне)

```
GIGA_TOKEN=...             # ключ GigaChat для разбора резюме нейросетью
GIGA_MODEL=GigaChat-Pro
WEB_CONCURRENCY=8          # число процессов бэкенда (≈ число CPU)
FSP_ID_MODE=mock           # mock | oidc | off
```

После изменения: `docker compose up -d --build backend`. Таблицу реальных результатов ФСП достаточно положить в `backend/data/fsp_results.xlsx` — пересборка не нужна; уже привязанным кандидатам достижения обновит `docker compose exec backend python -m scripts.fsp_data sync`.

## 10.6 Развёртывание на сервере (боевой режим)

1. Переменные `ENVIRONMENT`, `CORS_ORIGINS`, `BACKEND_PUBLIC_URL`, `FRONTEND_URL`, `SMTP_HOST` заданы в `docker-compose.yml` напрямую и через `.env` не переопределяются — для боевого режима отредактируйте секцию `backend.environment` (или используйте override-файл). Задать `ENVIRONMENT=prod`, длинный случайный `JWT_SECRET` (≥ 32 символов), `DATA_ENCRYPTION_KEYS` (ключ генерирует `python -m scripts.encryption gen-key`), `CORS_ORIGINS` без `*`, `BACKEND_PUBLIC_URL` и `FRONTEND_URL` на HTTPS-домен. При небезопасной конфигурации сервер не стартует.
2. Поставить перед контейнером frontend обратный прокси с TLS (nginx/Caddy). Чтобы лимиты считались по реальному IP клиента, во frontend-nginx добавить `set_real_ip_from <адрес прокси>; real_ip_header X-Forwarded-For;` — иначе все клиенты будут видны с адреса прокси (бэкенд доверяет последнему значению X-Forwarded-For).
3. Указать реальный SMTP вместо Mailpit: `SMTP_HOST` и добавить в `backend.environment` `SMTP_USER`, `SMTP_PASSWORD`; при необходимости — боевой ФСП ID (раздел 7.2).
4. Ключи шифрования задавать **до** первого запуска seed; ротация — `python -m scripts.encryption rotate`, проверка — `check`.

## 10.7 Запуск без Docker (для разработки)

```
# бэкенд: нужен PostgreSQL 16 и Python 3.12
cd backend && python -m venv .venv && . .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt
cp .env.example .env           # указать DATABASE_URL
alembic upgrade head && python -m scripts.seed
uvicorn app.main:app --reload --port 8000
# фронтенд — в другом терминале, из корня репозитория (Node.js 22)
cd frontend && npm ci && npm run dev   # http://localhost:5173, /api проксируется на :8000
```

## 10.8 Если что-то пошло не так

| Симптом | Решение |
|---|---|
| «port is already allocated» | порт занят другим проектом или локальным сервисом: остановить его (`docker ps`, `docker stop`) и повторить |
| Сайт открылся, но вход не работает | не запущен seed — выполнить `docker compose exec backend python -m scripts.seed` |
| Ошибка 502 сразу после старта | бэкенд ещё применяет миграции — подождать 10–20 секунд |
| Письмо подтверждения не пришло | все письма стенда приходят в Mailpit (порт 8025 / демо-почта стенда) |
| Нужна чистая база | `docker compose exec backend python -m scripts.seed --reset` или `docker compose down -v` |

# 11 Безопасность и 152-ФЗ

## 11.1 Аутентификация и защита от несанкционированного доступа

- Пароли — `scrypt` (N = $2^{14}$, r = 8, p = 1, соль 16 байт); политика: от 8 символов, буквы и цифры, запрет распространённых паролей и пароля, содержащего email.
- JWT HS256 с проверкой подписи, срока, издателя и типа; refresh-токены хранятся только как идентификатор, ротация с обнаружением повторного использования; выход отзывает одну сессию или все.
- Подтверждение email обязательно для входа (ссылка действует 48 часов). В режиме `dev` (по умолчанию в docker-compose) токен подтверждения для демонстрации возвращается в ответе регистрации; на публичном стенде нужен `ENVIRONMENT=prod`.
- Защита от перебора: 20 попыток входа в минуту с IP; после 5 неудачных попыток на один email — блокировка на 15 минут; при входе одинаковый ответ для существующих и несуществующих email; лимиты регистрации и повторных писем. Лимиты считаются в памяти каждого процесса uvicorn, поэтому при нескольких процессах фактические пороги пропорционально выше (общее хранилище Redis — в плане).
- Разграничение прав по ролям на сервере; чужие объекты — 404. Заголовки безопасности: `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Content-Security-Policy` и `Cache-Control: no-store` для API, HSTS в боевом режиме.
- Контейнер бэкенда работает не от root; база данных наружу не публикуется; все порты стенда привязаны к 127.0.0.1.

## 11.2 Персональные данные

- **Шифрование при хранении:** AES-256-GCM для ФИО, телефона, Telegram, контактного email и текста резюме кандидата; контактов работодателя; способа связи и ответа в приглашениях; секрета ATS; тела событий тестового приёмника. Поддерживается ротация ключей.
- **Согласия:** согласие на обработку — обязательная галочка в форме регистрации, на публикацию профиля — в настройках; без обоих согласий кандидат не попадает в выдачу. Время согласия сохраняется.
- **Минимизация:** до появления основания работодатель видит имя в виде «Имя Ф.» и не видит контактов; кандидат сам выбирает, показывать ли полное имя, город, «о себе» и достижения ФСП.
- **Основание раскрытия контактов:** только принятое приглашение этого работодателя (кандидат может отозвать доступ) или собственный отклик кандидата на вакансию этого работодателя.
- **Прозрачность:** журнал «кто и когда получил доступ к моим контактам» — фиксируются открытие карточки кандидата работодателем и передача данных в ATS (показ контактов в выдаче и избранном пока не журналируется).
- **Права субъекта:** выгрузка всех своих данных в JSON и удаление аккаунта кандидата с каскадным удалением связанных данных (с подтверждением паролем). Для аккаунтов, созданных через ФСП ID (без пароля), и для работодателей удаление через интерфейс пока не реализовано.
- **Передача третьим лицам:** перед отправкой текста резюме в GigaChat из него вырезаются email, телефоны и Telegram; используется российская модель. В клиенте GigaChat сейчас отключена

проверка TLS-сертификата (`verify_ssl_certs=False`) — для боевого контура её нужно включить с сертификатом НУЦ Минцифры. Журнал входов хранит email только как HMAC-отпечаток.

> Перечисленные механизмы — техническая основа соответствия 152-ФЗ; для промышленного запуска дополнительно оформляются политика обработки ПДн, сроки хранения и договоры с обработчиками.

# 12 Использование ML и LLM

По требованию организаторов (Q&A) описываем, какие модели используются, как и зачем. GPU не требуется; все модели либо локальные и лёгкие, либо облачные и необязательные.

| Задача | Модель / метод | Как работает |
|---|---|---|
| Разбор PDF-резюме | **GigaChat** (Сбер, российская LLM), модель GigaChat-Pro | pypdf извлекает текст первых 10 страниц (до 5 МБ) → из текста вырезаются контакты → запрос к GigaChat с просьбой вернуть только JSON: ФИО, навыки, стаж, направление и грейд (только из допустимых значений); навыки приводятся к справочнику. Таймаут 15 с. Используется только при заданном `GIGA_TOKEN` |
| Резервный разбор | правила и регулярные выражения | словарь навыков, оценка направления и грейда по навыкам и стажу, ФИО, город, контакты. Срабатывает без ключа или при любой ошибке LLM — демонстрация не зависит от облака |
| Подсказки, а не решения | — | распознанные данные заполняют пустые поля формы, навыки дополняют список; в интерфейсе сохраняются только после подтверждения кандидатом (в API по умолчанию `apply=true` — сразу); предполагаемые направление и грейд в профиль не записываются — категорию присваивает только тест |
| Текстовая близость в подборе | покрытие токенов (локально) | доля слов описания задачи, найденных в профиле и резюме; миллисекунды на кандидата, без сетевых запросов; вес 0.05. Альтернативная ветка — TF-IDF с косинусной близостью |
| Объяснения выдачи | правила | причины формируются детерминированно по факторам формулы — воспроизводимо и проверяемо |
| Тестирование | LLM не используется | задания генерируются серверными функциями банка; LLM во время теста не вызывается |

# 13 Дополнительный функционал

## 13.1 Публикация вакансий и самостоятельный отклик

Работодатель публикует вакансию с обязательной вилкой ЗП (gross/net), редактирует и закрывает её, получает подборку кандидатов под вакансию. Кандидат видит ленту активных вакансий с фильтрами (направление, грейд, поиск по тексту, минимальная зарплата, формат работы), откликается с сопроводительным письмом и может отозвать отклик. Статусы отклика: *отправлен* → *просмотрен* → *принят* / *отклонён*, *отозван*. Пока отклик не отозван, работодатель видит контакты откликнувшегося.

## 13.2 Регулярные короткие задания

Работодатель создаёт задание («решение» или «подход») для направления и, при желании, грейда, с дедлайном. Кандидат с подтверждённой категорией видит задания своей категории, отправляет одно решение на задание. Работодатель оценивает решение от 0 до 10 с комментарием. Средняя оценка решений (при наличии хотя бы одного) входит в фактор активности рейтинга (вес 0.10) — профиль поддерживается актуальным.

## 13.3 Интеграция с ATS

- Работодатель указывает адрес вебхука своей ATS; секрет подписи выдаётся один раз и хранится зашифрованным.
- События: `invitation.accepted` (кандидат принял приглашение — в ATS уходит карточка кандидата с контактами, категорией, баллом теста и ФСП) и `ats.test`.
- Подпись: `X-FSP-Signature: sha256=HMAC-SHA256(secret, timestamp + "." + body)`, плюс `X-FSP-Event-Id` для идемпотентности и `X-FSP-Timestamp` против повторной отправки.
- Защита от SSRF в боевом режиме: только HTTPS и запрет внутренних адресов. Журнал доставок виден работодателю.
- Встроенный тестовый приёмник `/api/mock-ats/webhook` проверяет подпись, возраст события (5 минут) и дубликаты — интеграцию можно показать без внешней ATS.

## 13.4 Концепция защиты от фиктивных вакансий и недобросовестных работодателей

| Мера | Статус |
|---|---|
| Обязательная вилка зарплаты в вакансии и приглашении (проверка API и ограничение в БД) — нельзя «собирать резюме» без конкретного предложения | **реализовано** |
| Контакты кандидата не раскрываются до его согласия; журнал доступа и отзыв доступа — фиктивная вакансия не даёт базы контактов | **реализовано** |
| Метка доверия компании: ИНН указан → «ИНН компании указан» в приглашении | **реализовано** |

| Мера | Статус |
| --- | --- |
| Лимиты частоты запросов и запрет повторного приглашения, пока предыдущее без ответа | **реализовано** |
| Проверка ИНН и статуса юрлица по данным ФНС (ЕГРЮЛ) с повышением метки до «проверена» | план |
| Подтверждение корпоративного домена email работодателя | план |
| Жалобы кандидатов и сигнальные метрики: много приглашений без принятий, вакансии без откликов, повторяющиеся тексты — очередь модерации | план |

## 13.5 Адаптивная версия для мобильных устройств

Интерфейс адаптирован для экранов до 640 px: навигация кабинета — сетка в две колонки, кнопки высотой не менее 44 px (удобно касаться), перенос элементов без горизонтальной прокрутки, поля ввода с шрифтом 16 px (без автоувеличения в iOS), модальные окна по высоте экрана. Редактор кода на телефоне переключается в упрощённый режим (крупный шрифт, без подсказок и сворачивания). Есть светлая и тёмная тема.

# 14 Ограничения и планы развития

## 14.1 Известные ограничения текущей версии

- Банк заданий: один шаблон на ячейку и 20 шаблонов с фиксированными данными (раздел 4.8).
- Исполнение кода изолировано на уровне процесса Python (AST-фильтр, отдельный процесс, лимиты, разделение эталонов), но не является промышленной песочницей уровня ОС.
- ФСП ID и реестр работают в режиме имитации; боевое подключение — настройкой (раздел 7.2).
- Лимиты частоты хранятся в памяти каждого процесса; вебхуки ATS отправляются одной попыткой без повторов.
- Валидация выполнена на синтетических данных (раздел 6).
- Метод предпросмотра заданий под вакансию открыт любой роли; реестр `GET /api/fsp/registry/{fsp_id}` доступен без входа — для боевого режима оба нужно закрыть.
- Нет восстановления и смены пароля; после отзыва доступа к контактам полное имя в списке приглашений работодателя остаётся видимым; если кандидат откликался на вакансию работодателя, контакты остаются видны по отклику.
- Демо-приёмник ATS и имитация ФСП ID работают только вне боевого режима (`ENVIRONMENT=prod` и `FSP_ID_MODE=oidc` их отключают).

## 14.2 План развития

| Этап | Что делаем |
|---|---|
| **Пилот** | подключение боевого ФСП ID и реестра результатов; изолированная песочница для кода (gVisor / Judge0 / nsjail); развёртывание с HTTPS и внешним SMTP; общий Redis для лимитов; повтор доставки вебхуков; пилот на участниках соревнований ФСП |
| **Рост** | расширение банка: несколько шаблонов на ячейку, рандомизация всех шаблонов, отдельные скрытые тесты; новые языки (JavaScript, Go, SQL) и направления; проверка компаний по данным ФНС; готовые интеграции с популярными ATS |
| **Масштаб** | калибровка порогов и весов на реальных наймах (P@k, nDCG, test-retest); прокторинг и антиплагиат; рекомендации кандидату «что подтянуть для следующего грейда»; аналитика для работодателей и вузов |

# П Приложения

## П.1 Демо-данные (`scripts/seed.py`)

Генерация детерминирована (Faker ru_RU, зерно 2026), тексты собраны из шаблонов с согласованием по роду. Баллы попыток считает настоящий движок тестирования, поэтому балл попытки, профиля и истории грейда совпадают.

| Объект | Количество |
|---|---|
| Компании (работодатели) | 6 (4 с ИНН) |
| Вакансии | 20: Backend 6, Frontend 4, Data Science 4, QA 3, DevOps 3 |
| Кандидаты | 200 + newbie; 50 с достижениями ФСП; 25 без пройденного теста (видны как «не подтверждён») |
| Приглашения | 26 в разных статусах |
| Отклики | до 30 |
| Короткие задания / решения | 6 / до 36 с оценками работодателей |

## П.2 Основные переменные окружения

| Переменная | По умолчанию | Назначение |
|---|---|---|
| ENVIRONMENT | dev | dev / prod / test; prod включает строгие проверки |
| DATABASE_URL | postgresql+psycopg://... | подключение к БД |
| JWT_SECRET | (dev-значение) | ключ подписи токенов; в prod — ≥ 32 символа |
| DATA_ENCRYPTION_KEYS | пусто (dev-ключ) | `kid:base64(32 байта)`, первый — текущий |
| ACCESS_TOKEN_MINUTES / REFRESH_TOKEN_DAYS | 15 / 14 | сроки жизни токенов |
| EMAIL_VERIFICATION_REQUIRED | true | вход только после подтверждения почты |
| TRUST_PROXY_HEADERS | false (в compose — true) | IP клиента из X-Forwarded-For (последнее значение) |
| TEST_PASS_THRESHOLD / TEST_UPGRADE_HINT_THRESHOLD | 70 / 90 | пороги теста |
| TEST_DURATION_MINUTES | 30 | длительность теста |
| GRADE_CHANGE_COOLDOWN_DAYS / TEST_RETRY_COOLDOWN_HOURS | 90 / 24 | ограничения частоты |
| RANK_W_TEST / FSP / SKILLS / TEXT / ACTIVITY | 0.35 / 0.15 / 0.35 / 0.05 / 0.10 | веса ранжирования |

| Переменная | По умолчанию | Назначение |
|---|---|---|
| RANK_UNVERIFIED_MULTIPLIER / RANK_ADJACENT_GRADE_MULTIPLIER | 0.6 / 0.7 | множители |
| FSP_ID_MODE, FSP_ID_ISSUER, FSP_ID_CLIENT_ID, FSP_ID_CLIENT_SECRET | mock, —, fsp-talent, (dev) | вход через ФСП ID |
| FSP_DATA_PATH / FSP_DEMO_FALLBACK | data/fsp_results.xlsx / true | реестр результатов ФСП |
| GIGA_TOKEN / GIGA_MODEL | — / GigaChat-Pro | разбор резюме нейросетью |
| SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD | —, 1025 | почта |
| WEB_CONCURRENCY | 4 | число процессов бэкенда |